

Security Audit Report

CIA Project — Web Application

Date 29 May 2026	Reviewed By Usama	Total Issues 22 found	Status All Fixed 
---------------------	----------------------	--------------------------	---



What This Report Is About

This report presents the findings of a thorough security review carried out on the CIA Project web application. The goal was to identify any weaknesses that could put the application — or its users — at risk.

Think of it like a home inspection: we went through every room looking for unlocked doors, broken locks, and safety hazards. We found 22 issues in total, ranging from serious problems that could have let attackers take full control of the system, to minor technical inconsistencies that posed little practical risk.

The good news: every single issue has been identified and fixed. The application is significantly more secure as a result of this review.

How to Read This Report

Each issue in this report includes three things:

- What was wrong** — A plain-language description of the problem that was found.
- Why it matters** — The real-world risk if this problem had been exploited by an attacker.
- How it was fixed** — What was done to close the gap.

Issues are grouped by severity — from Critical (fix immediately) down to Low (minor, low-risk). The colour on the left edge of each card matches the severity level.

Critical Issues

These issues posed the highest risk and would have been the first things an attacker would look for. All three have been resolved.

C-1 · Weak Secret Key		● CRITICAL
What was wrong	The application used a very short, easy-to-guess secret code to sign login tokens — like locking your front door with a key that thousands of other people also have.	
Why it matters	An attacker who discovered this secret could create their own fake login tokens, letting them access any account in the system — including admin accounts — without knowing anyone's password.	
How it was fixed	The secret has been replaced with a long, randomly generated key that is kept securely outside the application code. The system now refuses to start at all if no secure key is provided.	
C-2 · Anyone Who Registered Became an Admin		● CRITICAL
What was wrong	When a new user signed up for an account, the system automatically gave them full administrator privileges — the highest level of access possible.	
Why it matters	Any person who registered an account could immediately access all administrator tools: view and delete other users, change system settings, and do anything else reserved for trusted staff.	
How it was fixed	New accounts now receive standard user access only. Only existing administrators can promote someone to admin status.	
C-3 · Passwords Were Being Saved in Log Files		● CRITICAL
What was wrong	Every time a user logged in, their password and login token were automatically written to a log file stored on the server — like keeping a notebook next to your door that records everyone's PIN as they enter.	
Why it matters	Anyone who could access those log files — whether a system administrator, a contractor, or an attacker who gained partial access — would instantly have a list of real user passwords.	
How it was fixed	Log files no longer record any sensitive information. Only basic, non-sensitive details (like the time of a request) are kept.	

High Severity Issues

These issues were serious and could have been exploited with moderate effort. All seven have been resolved.

H-1 · Unlimited Login Attempts Allowed		▲ HIGH
What was	There was no limit on how many times someone could try to log in. An attacker could	

wrong	try thousands of passwords automatically without being stopped.
Why it matters	This is known as a "brute force" attack. Given enough time, an automated tool could guess weak passwords and break into user accounts.
How it was fixed	Login attempts are now limited to 10 tries every 15 minutes per IP address. After that, the system temporarily blocks further attempts.

H-2 · The Application Accepted Requests from Any Website

▲ HIGH

What was wrong	The application had no restrictions on which websites could send requests to it. Any site on the internet could silently make requests on behalf of a logged-in user.
Why it matters	A malicious website could trick a logged-in user into unknowingly performing actions — such as changing their password or deleting data — without the user realising anything happened.
How it was fixed	The application now only accepts requests from a specific, approved list of trusted websites.

H-3 · Attackers Could Overload the Server with Large Requests

▲ HIGH

What was wrong	There was no limit on the size of data that could be sent to the server in a single request. Someone could deliberately send enormous amounts of data repeatedly.
Why it matters	This is called a denial-of-service attack. The server would run out of memory trying to process these oversized requests, causing the application to crash or become unavailable for real users.
How it was fixed	The server now rejects any request that exceeds a reasonable data size limit.

H-4 · Database Could Be Accidentally Wiped on Restart

▲ HIGH

What was wrong	A setting was left on that told the database to automatically restructure itself every time the application restarted. This is a convenience feature meant only for development and testing.
Why it matters	If this setting had run in a live environment, it could silently delete or alter database columns — permanently destroying real user data with no warning and no way to undo it.
How it was fixed	This automatic restructuring is now disabled in live environments. Database changes must be applied deliberately and carefully.

H-5 · Login Tokens Were Stored Insecurely in the Browser

▲ HIGH

What was wrong	After logging in, the token that keeps a user's session active was stored in a way that any piece of code running in the browser could read it.
Why it matters	If an attacker managed to inject malicious code into the page (a very common type of attack), they could steal this token and use it to impersonate the user — taking over their session completely.
How it was fixed	Tokens are now stored with security flags that prevent any browser-side code from reading them directly. They are also restricted from being sent to other websites.

H-6 · Login Tokens Were Sent in an Unusual Way**▲ HIGH**

What was wrong	The application was sending login tokens using a non-standard, custom method instead of the widely recognised standard.
Why it matters	Many security tools, firewalls, and network monitors are designed to detect and protect against attacks on the standard method. A non-standard method flies under the radar and bypasses these protections.
How it was fixed	All tokens are now sent using the widely recognised industry standard, ensuring compatibility with security tools and best practices.

H-7 · User List Exposed Encrypted Passwords**▲ HIGH**

What was wrong	When the list of users was retrieved, the response included each user's stored (encrypted) password alongside their other details.
Why it matters	Even though passwords are stored in encrypted form, a determined attacker can use specialised software to crack common passwords from their encrypted versions. Exposing this data unnecessarily is a serious risk.
How it was fixed	The user list now only returns non-sensitive information such as usernames and account roles. Passwords — even in encrypted form — are never included in any response.

Medium Severity Issues

These issues were meaningful risks that required attention, but were less immediately exploitable than critical or high findings. All seven have been resolved.

M-1 · Default Admin Account Used "admin" as the Password**◆ MEDIUM**

What was wrong	The first administrator account was created automatically with the username "admin" and the password "admin" — one of the most commonly tried combinations in the world.
Why it matters	Automated hacking tools routinely try this combination as their very first attempt when targeting a new system. An attacker could have gained full admin access in seconds.
How it was fixed	The admin account is no longer set up with a fixed password. A strong, custom password must now be provided before the system can start.

M-2 · Password Encryption Was Set Too Weak**◆ MEDIUM**

What was wrong	Passwords were being encrypted using a setting that is considered outdated. Modern computers can process this level of encryption very quickly.
Why it matters	If the database were ever compromised, attackers could crack the stored passwords much faster than they should be able to — potentially exposing thousands of accounts.
How it was fixed	The encryption strength has been increased to the current industry standard, making passwords significantly harder and slower to crack even if the database were stolen.

M-3 · Weak Passwords Were Allowed

◆ MEDIUM

What was wrong	Users could set passwords as short as 4 characters, with no requirement for numbers, capital letters, or symbols. A password like "pass" was perfectly acceptable.
Why it matters	Short, simple passwords are very easy to guess — either by automated tools or even a person who knows something about the user. This puts all user accounts at risk.
How it was fixed	Passwords must now be at least 8 characters long and include a mix of uppercase letters, lowercase letters, numbers, and symbols.

M-4 · Login Tokens Were Being Printed in Internal Logs

◆ MEDIUM

What was wrong	Debug messages left in the application were printing users' login tokens — like printing someone's door key — into internal server logs every time a request was made.
Why it matters	Anyone with access to those server logs could extract active login tokens and use them to impersonate real users, without needing their password at all.
How it was fixed	All debug messages have been removed. Internal logs no longer contain any sensitive user information.

M-5 · Internal Monitoring Tools Were Publicly Accessible

◆ MEDIUM

What was wrong	Diagnostic pages and internal monitoring dashboards — meant only for developers — were accessible to anyone who knew the right web address, with no login required.
Why it matters	These pages revealed detailed information about how the application works internally: which pages are visited most, what errors occur, and how the system behaves. This is a goldmine of information for an attacker planning a targeted attack.
How it was fixed	These tools are now completely hidden and inaccessible in the live environment. They are only available in private development setups.

M-6 · The Registration Form Could Be Submitted Incompletely

◆ MEDIUM

What was wrong	A logic error in the registration form meant it could sometimes be submitted even if required fields were left blank or passwords didn't match.
Why it matters	This could lead to broken accounts being created in the system — accounts with missing information that might behave unpredictably or cause errors elsewhere in the application.
How it was fixed	The form validation has been fully rewritten. The form can no longer be submitted unless all required fields are correctly filled in and both password fields match.

M-7 · It Was Possible to Remove All Administrators

◆ MEDIUM

What was wrong	There was nothing stopping an administrator from accidentally (or intentionally) removing admin privileges from every admin account, including their own.
Why it matters	This would leave the application with no administrators at all — no one able to manage users, approve changes, or perform any admin-only action. Recovery would require direct access to the database.

How it was fixed	The system now prevents the last remaining administrator from being demoted. There must always be at least one admin account active.
------------------	--

Low Severity Issues

This issue posed minimal practical risk but was corrected as part of good housekeeping.

L-1 · Minor Data Type Mismatch When Deleting Users ▼ LOW	
What was wrong	When deleting a user account, the system was passing the user's ID in the wrong format — like writing a flat number as a word instead of a numeral.
Why it matters	In most cases this worked fine, but it could behave unexpectedly depending on the database being used, potentially failing silently or causing subtle errors.
How it was fixed	The user ID is now correctly converted to the right format before being used to delete a record.

Additional Improvements Made

Beyond fixing the identified issues, several proactive improvements were made to strengthen the application further.

I-1 · Stolen Login Tokens Could Not Be Cancelled ★ IMPROVEMENT	
What was wrong	Once a user logged in, their session token was valid for a full hour with no way to cancel it. There was also no real "log out" — logging out only deleted the token from the browser, but the token itself remained valid.
Why it matters	If an attacker stole someone's login token, they could continue using it for up to an hour with no way to stop them — even if the user changed their password.
How it was fixed	A proper two-part login system has been implemented. Short-lived tokens (valid for 15 minutes) handle active sessions, while a separate long-lived token handles silent re-login. Logging out now genuinely cancels the session on the server. Changing a password immediately cancels all active sessions across all devices.

I-2 · All Data Was Transmitted Without Encryption ★ IMPROVEMENT	
What was wrong	The application was running over plain, unencrypted connections — like sending a postcard instead of a sealed letter. Anyone on the same network could read the data passing between the browser and the server.
Why it matters	Passwords, login tokens, and personal data were all potentially visible to anyone on the same Wi-Fi or network — at a café, office, or anywhere else with shared internet access.
How it was	The application now uses encrypted connections (HTTPS) for all communication

fixed between the browser and the server. All data is protected in transit.

B-1 · Wrong Password Caused the Page to Reload Unexpectedly

✦ BUG

What was wrong	When a user typed an incorrect password, instead of simply showing an error message, the entire page would reload and the error would disappear — leaving the user confused about what happened.
Why it matters	This was a poor user experience that made the application feel broken. Users might not realise their password was wrong, leading to repeated failed attempts and frustration.
How it was fixed	The login form now correctly displays an error message when the password is wrong, without any unexpected page reloads.

A-1 · Regular Users Could Not View the User Directory

⬡ ACCESS CONTROL

What was wrong	The page that lists all users in the system was blocked for everyone except administrators, even though viewing the list poses no security risk.
Why it matters	Regular logged-in users saw a broken page when navigating to the users section, even though it was visible in the menu. This was an unnecessary restriction that created a confusing experience.
How it was fixed	Regular users can now view the user list. Only administrators retain the ability to make changes such as promoting or removing users.

Summary of All Issues

ID	Severity	Description	Fixed
C-1	Critical	Weak secret key — fake login tokens possible	✓
C-2	Critical	New registrations automatically received admin access	✓
C-3	Critical	Passwords were being written to server log files	✓
H-1	High	No limit on login attempts — brute-force attacks possible	✓
H-2	High	Application accepted requests from any website	✓
H-3	High	No data size limit — server could be overloaded	✓
H-4	High	Database could be wiped automatically on restart	✓
H-5	High	Login tokens stored insecurely in the browser	✓
H-6	High	Login tokens sent using a non-standard method	✓
H-7	High	User list exposed encrypted passwords	✓
M-1	Medium	Default admin account used "admin" as password	✓
M-2	Medium	Password encryption was set too weak	✓
M-3	Medium	Weak passwords (as short as 4 characters) were allowed	✓
M-4	Medium	Login tokens were being printed in internal server logs	✓
M-5	Medium	Internal monitoring tools were publicly accessible	✓

ID	Severity	Description	Fixed
M-6	Medium	Registration form could be submitted with empty fields	✓
M-7	Medium	All administrators could be removed, locking out the system	✓
L-1	Low	Minor data type mismatch when deleting user accounts	✓
I-1	Improvement	Stolen login tokens had no way to be cancelled remotely	✓
I-2	Improvement	All data transmitted without encryption (no HTTPS)	✓
B-1	Bug	Wrong password caused unexpected page reload instead of error	✓
A-1	Access Control	Regular users could not view the user directory	✓

Conclusion

This security review identified 22 issues across all severity levels. Every issue has been addressed and resolved. The application has been meaningfully hardened against the most common and serious categories of attack.

While no application can ever be declared completely risk-free, the CIA Project now follows current industry security standards and is in a significantly stronger position than it was before this review.